

四電一丙 數位邏輯實習

第 4 次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號: 114310326 黃子軒

組員 1 於此作業中的貢獻:解題、digital、Quartus、協助隊友排錯、拍照(作業二)、作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻:解題、digital、quartus、拍照(作業一)、作業報告(初步編排、複審內容)

I. 實驗目的

A. 作業 1

利用 VHDL 實作 2 位元的有號數乘法器，負數部分用 2 的補數表示，本題先導出最簡化的布林代數式，之後用 VHDL 實作電路，電路必須使用 VHDL 撰寫，並於報告中解釋程式的每個區塊。

B. 作業 2

利用 VHDL 實作一個優先編碼器（priority encoder），電路具有四個輸入（ x_3, x_2, x_1, x_0 ）及三個輸出（ z, y_1, y_0 ），當任一輸入（ x_3, x_2, x_1, x_0 ）為 1 時，則輸出 $z=1$ ，而 y_1 與 y_0 的輸出等於「輸入值為 1 的最大（優先權最高）編號」。本題先導出最簡化的布林代數式，之後用 VHDL 實作電路，電路必須使用 VHDL 撰寫，並於報告中解釋程式的每個區塊。

II. 設計程序

A. 作業 1

2-bit 有號數，因此輸入端為 -2~1，相乘後輸出為 -2~4，因此會需要 4bits 才足夠描述(1bit 作為正負符號判斷)

1. 依據投影片提供的範例建立真值表(輸入：X(x_1, x_0)、Y(y_1, y_0))；輸出：M(m_3, m_2, m_1, m_0))，運用 Digital 推導出 SOP 電路，找到輸出的最簡布林代數式。

表 1. 2-bit 有號數乘法器真值表

x_1	x_0	y_1	y_0	m_3	m_2	m_1	m_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	1	1	1	0
0	1	1	1	1	1	1	1
1	0	0	0	0	0	0	0
1	0	0	1	1	1	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	0	1	0
1	1	0	0	0	0	0	0
1	1	0	1	1	1	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

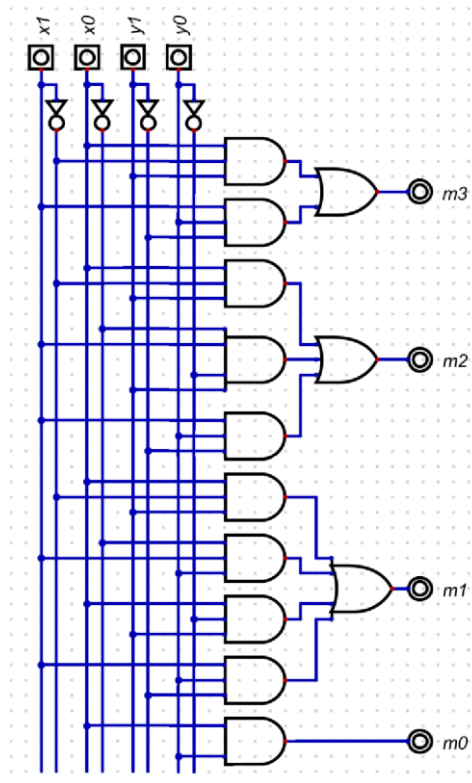


圖 1. 2-bit 有號數乘法器 SOP 電路

$$\begin{aligned}
 m3 &= (x0 \cdot \overline{x1} \cdot y1) + (x1 \cdot y0 \cdot \overline{y1}) \\
 m2 &= (x0 \cdot \overline{x1} \cdot y1) + (\overline{x0} \cdot x1 \cdot \overline{y0} \cdot y1) + (x1 \cdot y0 \cdot \overline{y1}) \\
 m1 &= (x0 \cdot \overline{x1} \cdot y1) + (\overline{x0} \cdot x1 \cdot y0) + (x0 \cdot \overline{y0} \cdot y1) + (x1 \cdot y0 \cdot \overline{y1}) \\
 m0 &= x0 \cdot y0
 \end{aligned}$$

圖 2. m3~m0 布林代數式

- 開啟新的分頁，運用 External 元件參考推得的 m3~m0 布林代數式編寫 VHDL，並使用 GHDL 進行編譯。

```

Program code:
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.all;
3 USE ieee.numeric_std.all;
4
5 entity multiplier is
6   port (
7     X1,X0,Y1,Y0: in std_logic;
8     M3,M2,M1,M0: out std_logic);
9 end multiplier;
10
11 architecture Equations of multiplier is
12 begin
13   -- concurrent assignment statements
14   M3 <= (x0 and not x1 and y1) or (x1 and y0 and not y1);
15   M2 <= (x0 and not x1 and y1) or (not x0 and x1 and not y0 and y1) or (x1 and y0 and not y1);
16   M1 <= (x0 and not x1 and y1) or (not x0 and x1 and y0) or (X0 and not y0 and y1) or (x1 and y0 and not y1);
17   M0 <= x0 and y0;
18 end Equations;

```

圖 3. 2-bit 有號數乘法器 VHDL

VHDL 程式碼講解：

紅色框 :是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。

橘色框 :是介面定義，定義有四個輸入 x1、x0、y1、y0，輸出也是四個 m3、m2、m1、m0，以及形態 (std_logic)。

黃色框 :程式的核心運算區，定義輸入與輸出間關係，如圖定義 m3~m0 布林代數式，各行程式間並無順序，模擬實際電路中邏輯閘關係。

3. 完成 External，測試不同輸入組合，確認輸出結果符合原先表示式後匯出 VHDL 檔案。

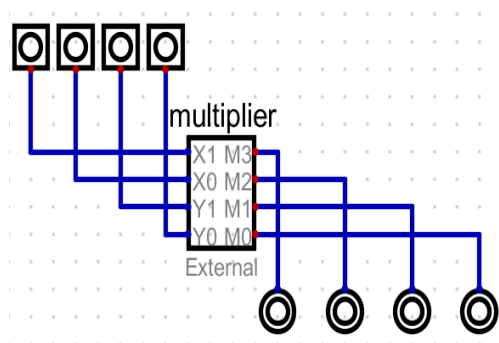


圖 4. 2-bit 有號數乘法器模擬電路

B. 作業 2

4-to-2 Priority Encoder，Z 可以用來區分是否有 1，Y 則是知道出現 1 的 MSB 編號，低於為 1 的 MSB 的輸入部分為 don't care。

1. 參考作業 1.設計流程，依據投影片提供的範例建立真值表(輸入：X(x3,x2,x1,x0)、輸出：z,y1,y0)，運用 Digital 推導出 SOP 電路，找到輸出的最簡布林代數式。(4-to-2 Priority Encoder：4 個輸入，2 個資料位元(y1,y0) +1 個狀態位元(z))

表 2.4 對 2 優先編碼器真值表

X3	X2	X1	X0	Z	Y1	Y0
0	0	0	0	0	0	0
0	0	0	1	1	0	0
0	0	1	0	1	0	1
0	0	1	1	1	0	1
0	1	0	0	1	1	0
0	1	0	1	1	1	0
0	1	1	0	1	1	0
0	1	1	1	1	1	0
1	0	0	0	1	1	1
1	0	0	1	1	1	1
1	0	1	0	1	1	1
1	0	1	1	1	1	1
1	1	0	0	1	1	1
1	1	0	1	1	1	1
1	1	1	0	1	1	1
1	1	1	1	1	1	1

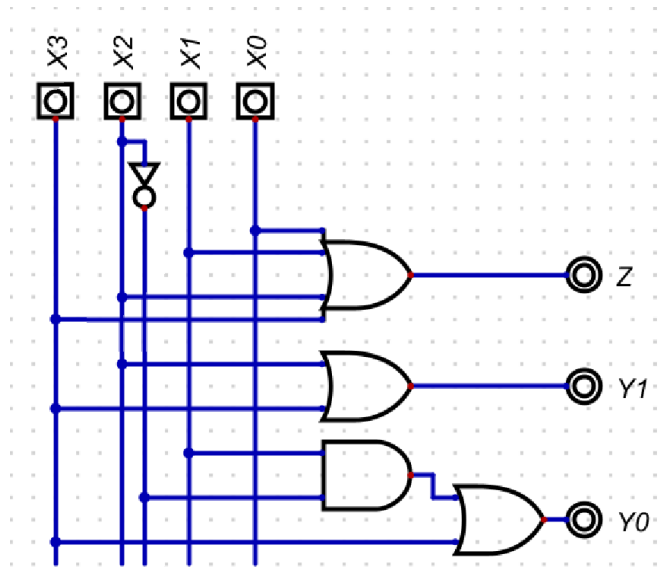


圖 5.4 對 2 優先編碼器 SOP 電路

$$\begin{aligned}
 Z &= X0 + X1 + X2 + X3 \\
 Y1 &= X2 + X3 \\
 Y0 &= (X1 \cdot \overline{X2}) + X3
 \end{aligned}$$

圖 6. z, y1, y0 布林代數式

- 開啟新的分頁，運用 External 元件參考推得的 z,y1,y0 布林代數式編寫 VHDL，並使用 GHDL 進行編譯。

```

1 LIBRARY ieee;
2 USE ieee.std_logic_1164.all;
3 USE ieee.numeric_std.all;
4
5 entity priorityencoder is
6   port (
7     X3, X2, X1, X0: in std_logic;
8     Z, Y1, Y0: out std_logic);
9 end priorityencoder;
10
11 architecture Equations of priorityencoder is
12 begin
13   -- concurrent assignment statements
14   Z <= X3 or X2 or X1 or X0;
15   Y1 <= X3 or X2;
16   Y0 <= X3 or ((not X2) and X1) ;
17 end Equations;

```

圖 7.4 對 2 優先編碼器 VHDL

VHDL 程式碼講解：(如作業 1)

紅色框：是 Library 宣告區，載入資料庫，定義邏輯閘功能與語法。

橘色框：是介面定義，定義有四個輸入 x_3 、 x_2 、 x_1 、 x_0 ，輸出是三個 z 、 y_1 、 y_0 ，以及形態(std_logic)。

黃色框：程式的核心運算區，定義輸入與輸出間關係，如圖定義 z 、 y_1 、 y_0 布林代數式，各行程式間並無順序，模擬實際電路中邏輯閘關係。

3. 完成 External，測試不同輸入組合，確認輸出結果符合原先表示式後匯出 VHDL 檔案。

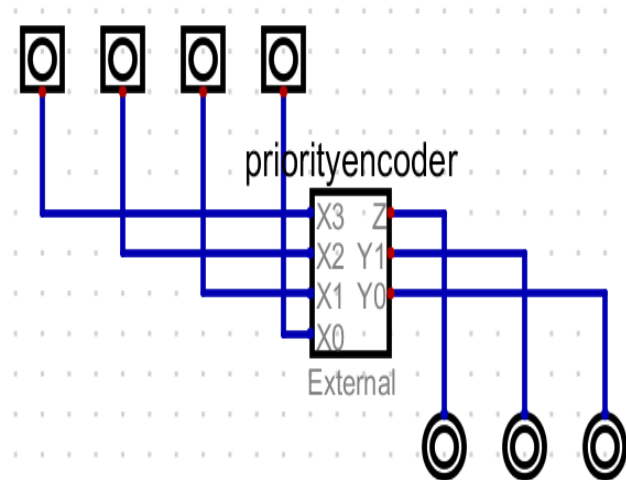


圖 8.4 對 2 優先編碼器模擬電路

III. 模擬驗證

A. 作業 1

2-bit 有號數乘法器：Input 由左至右分別為 $X(x_1, x_0)$ 、 $Y(y_1, y_0)$ ；Output $M(m_3, m_2, m_1, m_0)$

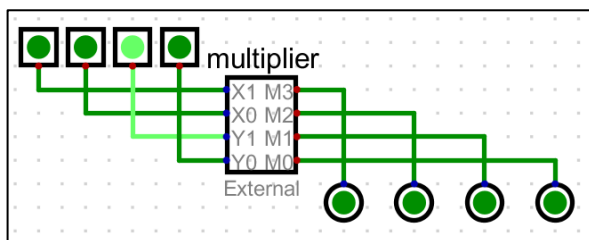


圖 9. Input: 0010 / Output: 0000
Decimal: $0 * -2 = 0$

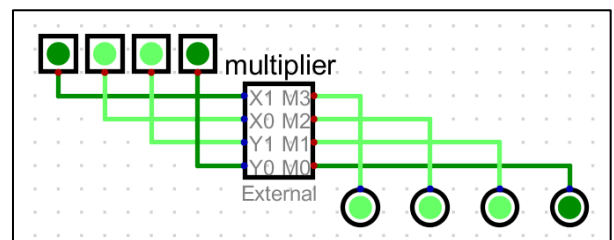


圖 10. Input: 0110 / Output: 1110
Decimal: $1 * -2 = -2$

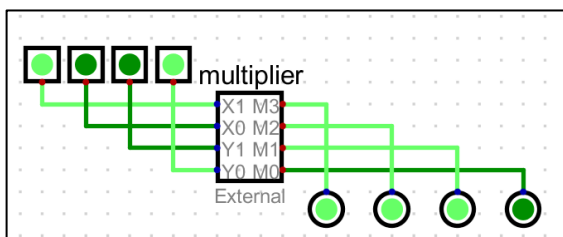


圖 11. Input: 1001 / Output: 1110
Decimal: $-2 * 1 = -2$

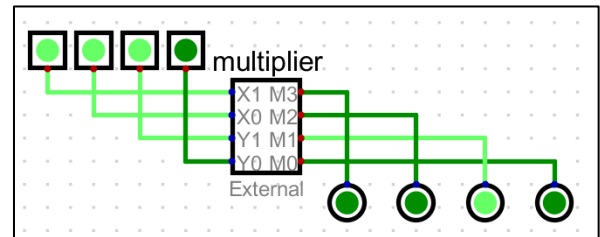


圖 12. Input: 1110 / Output: 0010
Decimal: $-1 * -2 = 2$

B. 作業 2

4 對 2 優先編碼器：Input $X(x_3, x_2, x_1, x_0)$ ；Output z, y_1, y_0

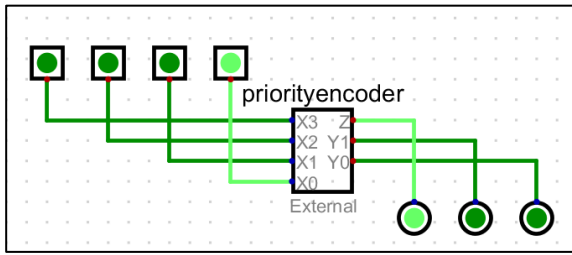


圖 13. Input:0001 / Output:100
有 1，為 1 的最大位元於 x0

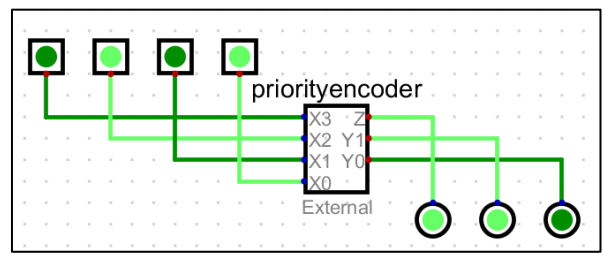


圖 14. Input:0101 / Output:110
有 1，為 1 的最大位元於 x2

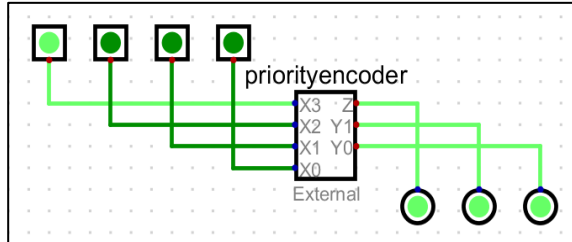


圖 15. Input:1000 / Output:111
有 1，為 1 的最大位元於 x3

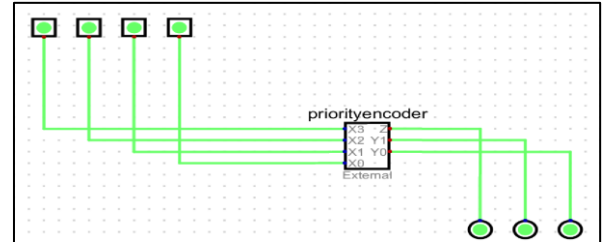


圖 16. Input:1111 / Output:111
有 1，為 1 的最大位元於 x3

IV. 實驗結果與成果討論

A. 作業 1

2-bit 有號數乘法器：Input 由左至右分別為 $X(x1, x0)$ 、 $Y(y1, y0)$ ；Output $M(m3, m2, m1, m0)$ 。

輸入 $x1\ x0\ y1\ y0$ ，為 $sw3$ 、 $sw2$ 、 $sw1$ 、 $sw0$ ，輸出 $m3\ m2\ m1\ m0$ ，為 $LED3$ 、 $LED2$ 、 $LED1$ 、 $LED0$ 。

(以下結果均符合模擬預測)

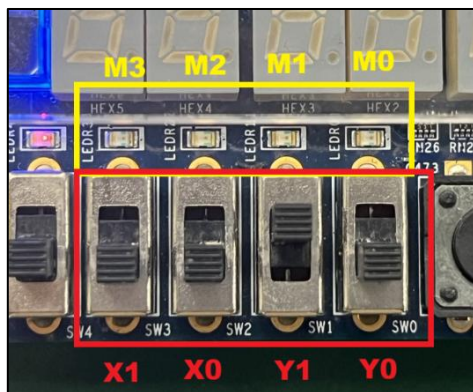


圖 17. Input:0010 / Output:0000
Decimal : $0 * -2 = 0$

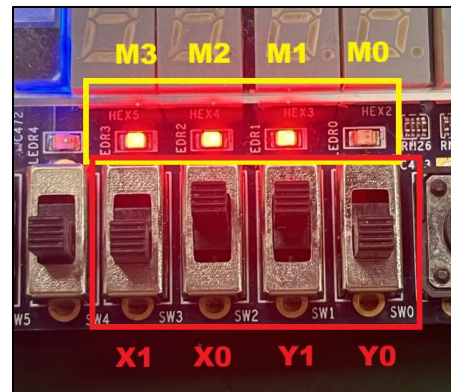


圖 18. Input:0110 / Output:1110
Decimal : $1 * -2 = -2$

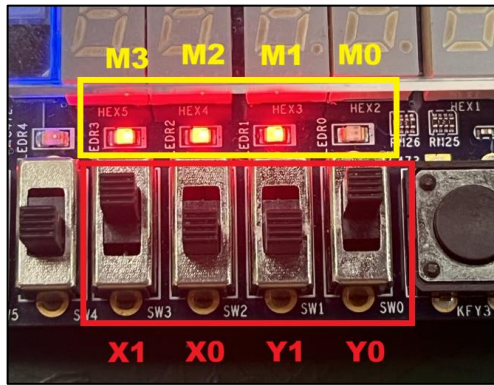


圖 19. Input:1001 / Output: 1110
Decimal : $-2 * 1 = -2$

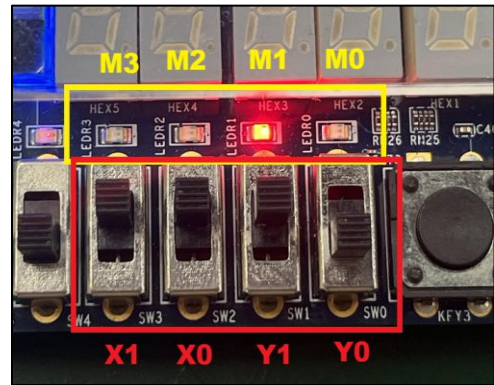


圖 20. Input:1110 / Output: 0010
Decimal : $-1 * -2 = 2$

B. 作業 2

4 對 2 優先編碼器：Input X(x3,x2,x1,x0)；Output z,y1,y0。

輸入 x3 x2 x1 x0，為 sw3、sw2、sw1、sw0 開關，輸出 z y1 y0，為 LED2、LED1、LED0。

(以下結果均符合模擬預測)

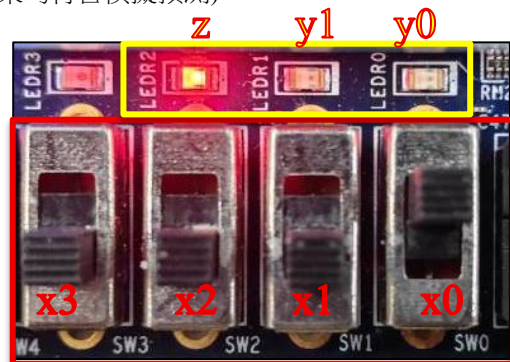


圖 21. Input:0001 / Output:100
有 1，為 1 的最大位元於 x0

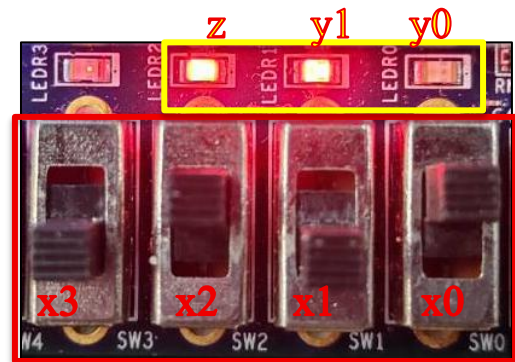


圖 22. Input:0101 / Output:110
有 1，為 1 的最大位元於 x2

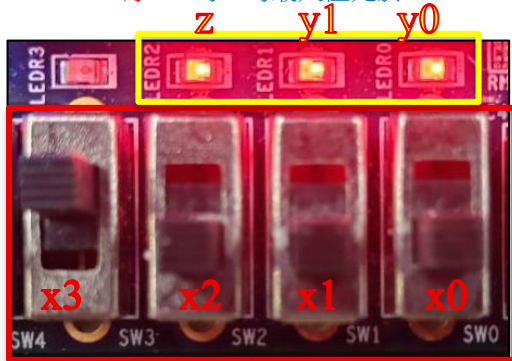


圖 23. Input:1000 / Output: 111
有 1，為 1 的最大位元於 x3

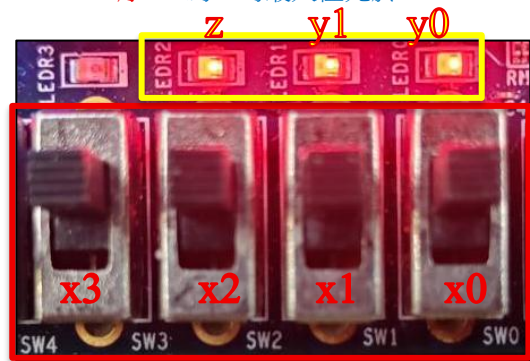


圖 24. Input: 1111 / Output: 111
有 1，為 1 的最大位元於 x3

V. 實驗心得

組員 1：與 Verilog 不同，VHDL 更加嚴謹，編程也得更加細心。這一次因為是初學 VHDL 以及換組適應，難度不會太高，實驗一也許可以試著加入七節顯示器的腳位把結果做得更漂亮。

組員 2：第一題的布林代數有一列打錯了，一開始沒有進行二次檢查，直到燒錄電路才發現，導致我們重新修正，多花了一點時間，之後都要做完一部分，檢查一次。

參考文獻

- [1] DLE 05 數位邏輯實習 VHDL(I)